Case Study 4: Examination Result and Grace-Mark Validation

1. Problem statement

Develop a Python program to calculate a student's examination result based on marks obtained in different subjects. The program should check whether the student passes or fails and determine whether grace marks can be awarded to help the student pass.

2. Objectives

- To accept marks from the user.
- To identify passed and failed subjects.
- To apply grace marks according to the specified rules.
- To display the final examination result.
- To understand loops, conditional statements, lists, and user input in Python.

3. Assumptions

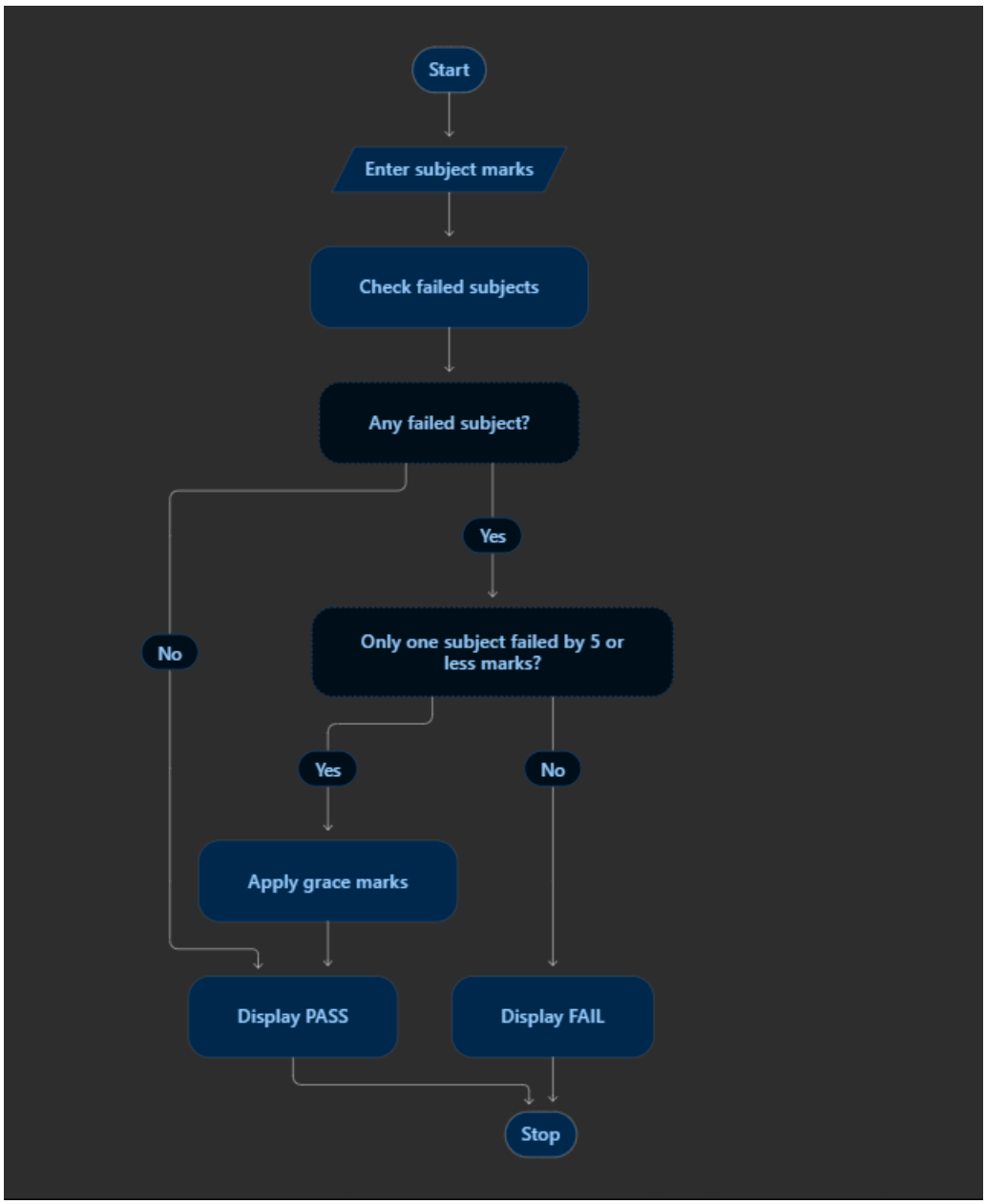
- The pass mark for each subject is 35 out of 100.
- Marks must be between 0 and 100.
- A maximum of 5 grace marks can be awarded.
- Grace marks can be awarded to only one subject.
- Grace marks are awarded only when the student fails in exactly one subject and needs 5 or fewer additional marks to pass.
- If these conditions are not satisfied, the student fails.

4. Algorithm

1. Start the program.
2. Accept the number of subjects.

3. Accept the marks obtained in each subject.
4. Check whether the marks are valid.
5. Count the subjects in which the student scored below 35.
6. If all subjects are passed, display PASS.
7. If exactly one subject is failed and the marks needed are 5 or fewer, apply grace marks.
8. Otherwise, display FAIL.
9. Display the marks and final result.
10. Stop the program.

5. Flowchart



6. Pseudocode

START

INPUT number_of_subjects

INPUT marks for each subject

SET pass_mark = 35

SET grace_limit = 5

COUNT failed subjects

IF no subjects are failed THEN

DISPLAY "PASS"

ELSE IF exactly one subject is failed AND

marks needed <= grace_limit THEN

ADD grace marks

DISPLAY "PASS with grace marks"

ELSE

DISPLAY "FAIL"

END IF

DISPLAY final marks

STOP

7. Source code

```
n = int(input("Enter number of subjects: "))

if n <= 0:
    print("Invalid number of subjects!")

else:
    marks = []
    valid = True

    for i in range(n):
        mark = int(input("Enter marks for subject " + str(i + 1) + ": "))

        if mark < 0 or mark > 100:
            print("Invalid marks!")
            valid = False
            break

        marks.append(mark)

    if valid:
        pass_mark = 35
        failed = []

        for i in range(n):
            if marks[i] < pass_mark:
                failed.append(i)

        if len(failed) == 0:
            print("Result: PASS")
```

```
        print("Result: PASS")

    elif len(failed) == 1:
        index = failed[0]
        grace_needed = pass_mark - marks[index]

        if grace_needed <= 5:
            marks[index] = pass_mark
            print("Grace marks awarded:", grace_needed)
            print("Result: PASS with grace marks")
        else:
            print("Result: FAIL")

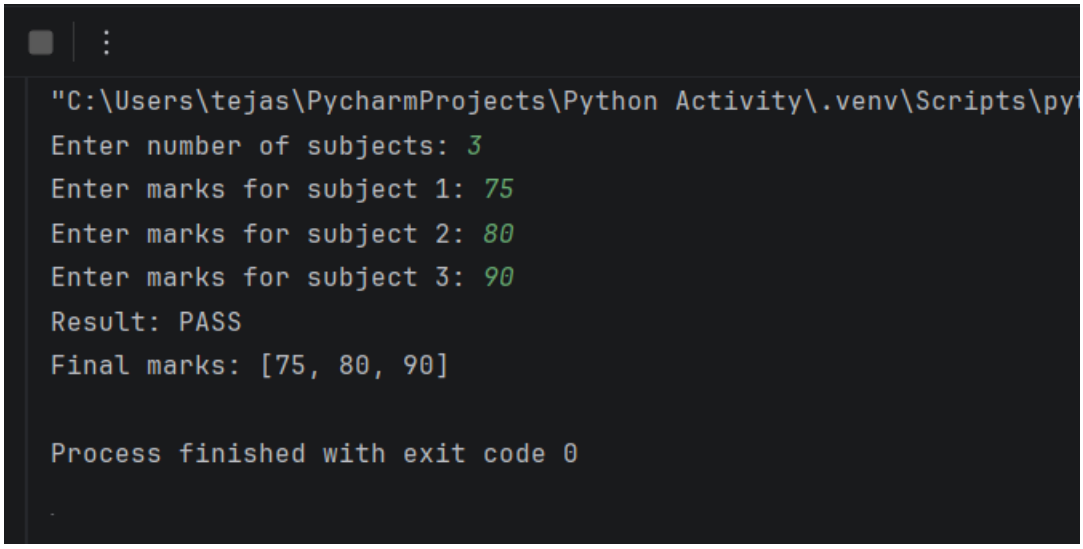
    else:
        print("Result: FAIL")

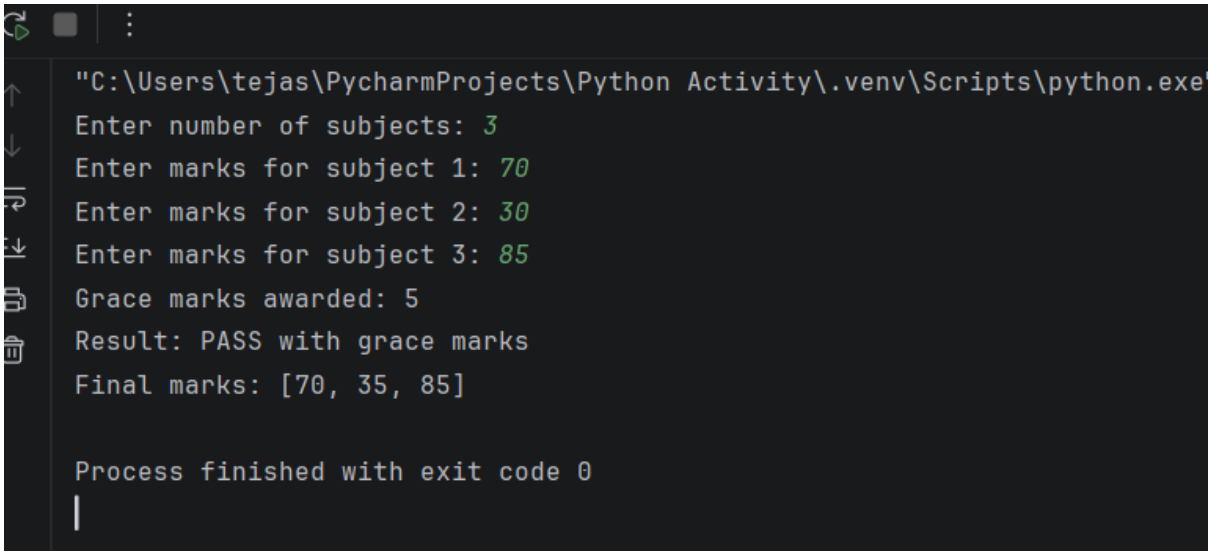
    print("Final marks:", marks)
```

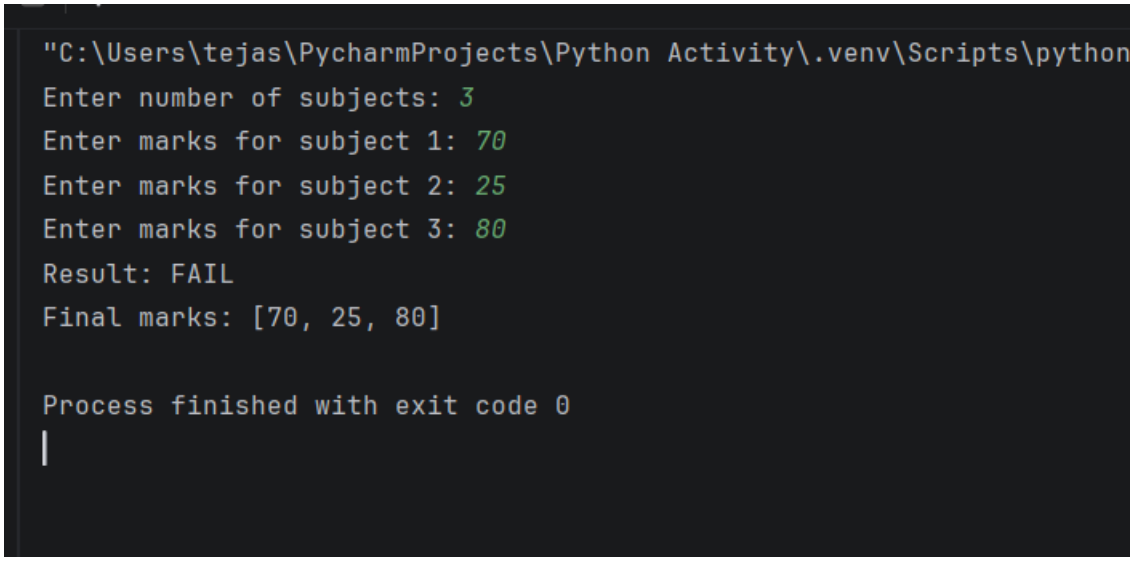
8. Test cases

Marks entered	Expected result
75, 80, 90	PASS
70, 30, 85	PASS with 5 grace marks
70, 25, 85	FAIL

9.Output


"C:\Users\tejas\PycharmProjects\Python Activity\.venv\Scripts\pyt
Enter number of subjects: 3
Enter marks for subject 1: 75
Enter marks for subject 2: 80
Enter marks for subject 3: 90
Result: PASS
Final marks: [75, 80, 90]
Process finished with exit code 0


"C:\Users\tejas\PycharmProjects\Python Activity\.venv\Scripts\python.exe
Enter number of subjects: 3
Enter marks for subject 1: 70
Enter marks for subject 2: 30
Enter marks for subject 3: 85
Grace marks awarded: 5
Result: PASS with grace marks
Final marks: [70, 35, 85]
Process finished with exit code 0


"C:\Users\tejas\PycharmProjects\Python Activity\.venv\Scripts\python
Enter number of subjects: 3
Enter marks for subject 1: 70
Enter marks for subject 2: 25
Enter marks for subject 3: 80
Result: FAIL
Final marks: [70, 25, 80]
Process finished with exit code 0

10. Conclusion

The Examination Result and Grace-Mark Validation program checks students' marks and determines their examination results. It awards grace marks when the specified conditions are satisfied. This program demonstrates the use of lists, loops, conditional statements, and user input in Python.